

# Proyecto 1 — Entrega parcial

**Curso:** CC3067 Redes **Universidad del Valle de Guatemala Repositorio:** <https://github.com/andresm220/Proyecto1Redes> **Servidor MCP desarrollado:** `netops` **v1.0.0 Versión de protocolo implementada:** `2025-11-25`

## 8. Especificación del servidor MCP desarrollado

La especificación completa, con los esquemas íntegros y los intercambios JSON-RPC crudos, está versionada en el repositorio en `servers/netops/SPEC.md`. Esta sección la resume y explica las decisiones detrás de ella.

### 8.1 Propósito y caso de uso

`netops` modela el back office de la mesa de soporte técnico de un proveedor de internet. Expone las operaciones que un agente de primera línea realiza mientras tiene al abonado en el teléfono: identificar la cuenta, leer el estado actual del enlace, verificar si una incidencia masiva ya explica el reclamo, correr un diagnóstico, y abrir o dar seguimiento a un ticket.

El caso de uso se eligió porque su valor está en el **encadenamiento**, no en una consulta aislada. Un reclamo de "no tengo internet" no se resuelve con una sola llamada: hay que identificar al abonado, leer su enlace y revisar las incidencias de su región antes de decidir si un ticket siquiera corresponde. Eso lo vuelve un caso apropiado para un host gobernado por un LLM, que puede encadenar las llamadas y razonar sobre los resultados.

Los datos son simulados pero **determinísticos**: las métricas del enlace (latencia, pérdida de paquetes, SNR) se derivan de un hash del identificador de cuenta en lugar de un generador aleatorio, de modo que la misma llamada devuelve siempre la misma lectura. Los tickets y visitas, en cambio, son estado persistente real que sobrevive al reinicio del proceso.

Los datos semilla están contruidos de forma que las entidades se relacionen entre sí. La cuenta `GT-10233` está en la región `peten`, donde la incidencia `OUT-2026-013` está activa; por eso `check_service_status` sobre esa cuenta reporta la incidencia en lugar de una falla individual. Sin esa relación, el modelo no tendría motivo para encadenar herramientas.

### 8.2 Transporte y protocolo

| Aspecto     | Valor                                                           |
|-------------|-----------------------------------------------------------------|
| Protocolo   | JSON-RPC 2.0                                                    |
| Versión MCP | <code>2025-11-25</code>                                         |
| Transporte  | <code>stdio</code> (el host lanza el servidor como subprocesso) |
| Framing     | NDJSON: un mensaje JSON por línea, terminado en <code>\n</code> |

| Aspecto      | Valor                                                           |
|--------------|-----------------------------------------------------------------|
| Codificación | UTF-8                                                           |
| stdout       | Exclusivamente mensajes de protocolo                            |
| stderr       | Exclusivamente logs del servidor, nunca parseado como protocolo |

**No se utilizó ningún SDK de MCP.** Los sobres JSON-RPC, el handshake, la correlación de peticiones y el bucle de despacho están implementados a mano en `host/mcp/jsonrpc.py` y `servers/netops/stdio_server.py`. La única dependencia de terceros en el servidor es ninguna: corre con la biblioteca estándar de Python, lo cual se verificó ejecutándolo con un intérprete sin el entorno virtual.

El framing es NDJSON y **no** el encabezado `Content-Length` que utiliza LSP. Un mensaje no puede contener saltos de línea embebidos; `json.dumps` escapa los saltos dentro de las cadenas, por lo que una carga útil con saltos de línea permanece en una sola línea.

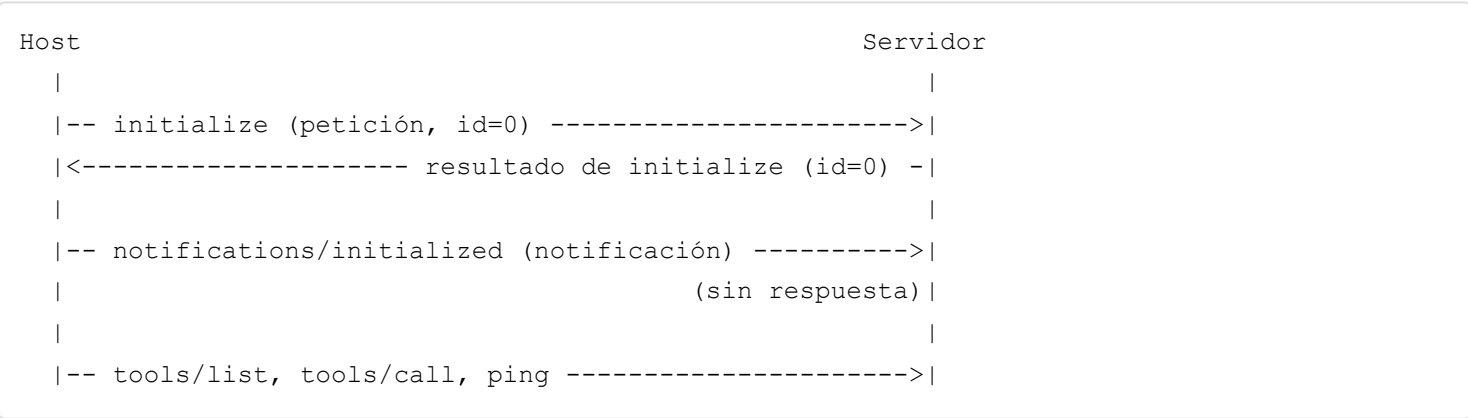
### 8.3 Endpoints

Sobre stdio no existen endpoints HTTP: no hay URLs ni rutas. El equivalente funcional son los **métodos JSON-RPC** que el servidor atiende, y el direccionamiento lo da el campo `method` del sobre en lugar de una ruta. Cuando el proyecto avance a la fase remota, esos mismos métodos viajarán sobre un único endpoint HTTP, y la lógica de negocio no cambiará porque vive en un módulo independiente del transporte.

| Método                                 | Tipo         | Requiere handshake | Descripción                                             |
|----------------------------------------|--------------|--------------------|---------------------------------------------------------|
| <code>initialize</code>                | petición     | —                  | Negocia la versión de protocolo e identifica al cliente |
| <code>notifications/initialized</code> | notificación | —                  | El cliente confirma el handshake. No se responde nunca  |
| <code>ping</code>                      | petición     | No                 | Verificación de vida                                    |
| <code>tools/list</code>                | petición     | Sí                 | Devuelve el catálogo de herramientas con sus esquemas   |
| <code>tools/call</code>                | petición     | Sí                 | Ejecuta una herramienta                                 |

Cualquier otro método devuelve `-32601`.

### Ciclo de vida



El handshake es obligatorio: `tools/list` y `tools/call` se rechazan con `-32600` hasta que se complete. Si el servidor respondiera una versión de protocolo que el cliente no soporta, el cliente debe desconectarse y reportarlo en lugar de continuar; esa rama está implementada y cubierta por pruebas.

Ejemplo de handshake (capturado de una ejecución real):

```
{"jsonrpc": "2.0", "id": 0, "method": "initialize", "params": {"protocolVersion": "2025-11-25", "capab...
{"jsonrpc": "2.0", "id": 0, "result": {"protocolVersion": "2025-11-25", "capabilities": {"tools": {"li
```

El campo `instructions` es orientación redactada para el modelo. El host la incorpora a su `system prompt`, que es exactamente el propósito para el que el protocolo la define.

### 8.4 Herramientas y parámetros

Siete herramientas. Los esquemas completos están en `servers/netops/SPEC.md` sección 4; aquí se listan los parámetros y su validación.

`lookup_account`

Busca un abonado por identificador de cuenta o por teléfono.

| Parámetro               | Tipo   | Obligatorio    | Restricciones             |
|-------------------------|--------|----------------|---------------------------|
| <code>account_id</code> | string | Uno de los dos | <code>minLength: 3</code> |
| <code>phone</code>      | string | Uno de los dos | <code>minLength: 8</code> |

Debe suministrarse **exactamente uno** de los dos. Los teléfonos se comparan solo por sus dígitos, de modo que `+502 5555-0101`, `50255550101` y `(502) 5555 0101` resuelven a la misma cuenta.

**Devuelve:** plan, velocidad contratada, estado de la cuenta, tecnología, región, dirección de servicio y fecha de instalación.

`check_service_status`

Lee el estado actual del enlace de una cuenta.

| Parámetro               | Tipo   | Obligatorio | Restricciones             |
|-------------------------|--------|-------------|---------------------------|
| <code>account_id</code> | string | Sí          | <code>minLength: 3</code> |

**Devuelve:** `link_state` (`up`, `degraded`, `down`, `not_provisioned`), `reason`, detalle legible, y un objeto `metrics` con latencia, pérdida de paquetes, SNR y velocidades medidas. Si una incidencia masiva cubre la región de la cuenta, incluye además `outage_id` y `eta`.

El campo `reason` se evalúa por prioridad: una suspensión administrativa pesa más que una incidencia masiva, y esta pesa más que una falla medida. Las métricas se devuelven siempre, incluso con el enlace caído, para que quien consulte pueda distinguir "sin señal" de "cuenta suspendida por saldo".

`list_outages`

Lista incidencias masivas con su causa y tiempo estimado de reparación.

| Parámetro   | Tipo    | Obligatorio | Restricciones                                        |
|-------------|---------|-------------|------------------------------------------------------|
| region      | string  | No          | enum: guatemala , quetzaltenango , peten , escuintla |
| active_only | boolean | No          | Por defecto true                                     |

**Devuelve:** conteo y arreglo de incidencias con `outage_id` , región, estado, causa, inicio, ETA, cuentas afectadas y servicios afectados.

Es la única herramienta sin modo de fallo de dominio: una región sin incidencias devuelve `count: 0` , que no es un error.

`run_diagnostic`

Ejecuta un diagnóstico contra la línea del abonado.

| Parámetro  | Tipo   | Obligatorio | Restricciones             |
|------------|--------|-------------|---------------------------|
| account_id | string | Sí          | minLength: 3              |
| test_type  | string | Sí          | enum: ping , speed , line |

**Devuelve:** un objeto `readings` cuyo contenido depende del tipo de prueba, más un campo `probable_cause` siempre presente.

| test_type | Campos de readings                                              |
|-----------|-----------------------------------------------------------------|
| ping      | latency_ms , jitter_ms , packet_loss_pct , packets_sent         |
| speed     | downstream_mbps , upstream_mbps , contracted_mbps , pct_of_plan |
| line      | snr_db , attenuation_db , technology , sync_errors_last_hour    |

`open_ticket`

Abre un ticket de soporte contra una cuenta.

| Parámetro   | Tipo   | Obligatorio | Restricciones                                                   |
|-------------|--------|-------------|-----------------------------------------------------------------|
| account_id  | string | Sí          | minLength: 3                                                    |
| category    | string | Sí          | enum: connectivity , speed , billing , equipment , installation |
| description | string | Sí          | minLength: 5 , maxLength: 2000                                  |
| priority    | string | No          | enum: low , normal , high , critical . Por defecto normal       |

Los identificadores son secuenciales y con relleno de ceros: `TCK-00001` . El ticket se escribe a disco **antes** de enviar la respuesta, por lo que un `get_ticket` posterior —en esta sesión o en otra— lo encuentra.

`get_ticket`

Lee el estado, el historial y la visita agendada de un ticket.

| Parámetro | Tipo   | Obligatorio | Restricciones |
|-----------|--------|-------------|---------------|
| ticket_id | string | Sí          | minLength: 5  |

**Devuelve:** el ticket completo, incluido un `history` de solo-anexado con entradas `{at, status, note}` .

`schedule_visit`

Agenda una visita técnica contra un ticket existente.

| Parámetro                | Tipo   | Obligatorio | Restricciones                                              |
|--------------------------|--------|-------------|------------------------------------------------------------|
| <code>ticket_id</code>   | string | Sí          | <code>minLength: 5</code>                                  |
| <code>date</code>        | string | Sí          | <code>pattern: ^\d{4}-\d{2}-\d{2}\$</code>                 |
| <code>time_window</code> | string | Sí          | <code>enum: 08:00-12:00 , 12:00-16:00 , 16:00-20:00</code> |

Agendar sobre un ticket que ya tiene visita la reagenda, y la respuesta lo indica con `"rescheduled": true`.

## 8.5 Formato de respuesta

Toda llamada exitosa devuelve el mismo sobre:

```
{"content": [{"type": "text", "text": "<carga útil JSON>"}], "isError": false}
```

El bloque `text` contiene un objeto JSON indentado. Un fallo de dominio usa la misma forma con `"isError": true` y una carga útil `{"error": "<mensaje>", ...contexto}`.

## 8.6 Manejo de errores

La distinción entre error de protocolo y error de herramienta gobierna todo el diseño, y es probablemente la decisión más importante del servidor.

Un **error de protocolo** significa que el intercambio se rompió: el mensaje no pudo parsearse, el sobre era inválido, el método no existe, o los argumentos no corresponden al esquema publicado. Se reporta en el campo `error` de JSON-RPC y no hay `result`.

Un **error de herramienta** significa que el intercambio estuvo bien y la operación no pudo completarse: una cuenta que no existe, un ticket que nunca se abrió, una fecha en el pasado. Se reporta como un `result` **exitoso** cuyo campo `isError` vale `true`. Se espera que el modelo lo lea y reaccione, de modo que es un dato, no una falla de transporte.

Colapsar ambos rompería el host en las dos direcciones: una cuenta inexistente parecería una conexión caída, y una petición malformada parecería un resultado de negocio sobre el cual el modelo debería razonar.

| Código | Nombre           | Se emite cuando                                                                                                                                                |
|--------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -32700 | Parse error      | Una línea de stdin no es JSON válido. El id no es recuperable, por lo que la respuesta lleva <code>"id": null</code> — el único caso donde un id nulo es legal |
| -32600 | Invalid Request  | JSON válido pero sobre inválido; también si se llama <code>tools/list</code> o <code>tools/call</code> antes del handshake                                     |
| -32601 | Method not found | Cualquier método fuera de los cinco soportados                                                                                                                 |
| -32602 | Invalid params   | Falta <code>params.name</code> , la herramienta no existe, o algún argumento falla la validación del esquema                                                   |

| Código | Nombre         | Se emite cuando                                                                |
|--------|----------------|--------------------------------------------------------------------------------|
| -32603 | Internal error | Excepción no controlada en un manejador. El traceback va a <code>stderr</code> |

La validación de argumentos ocurre **antes** de ejecutar cualquier manejador, de modo que un argumento inválido nunca alcanza la lógica de negocio. Como `jsonschema` no está entre las dependencias permitidas por el enunciado, se implementó a mano el subconjunto de JSON Schema que los esquemas utilizan: `type`, `required`, `enum`, `minLength`, `maxLength`, `pattern`, `minimum`, `maximum` y `additionalProperties`.

El parámetro `date` de `schedule_visit` ilustra los dos lados de la frontera con el mismo argumento:

| Entrada      | Resultado                  | Razón                                                   |
|--------------|----------------------------|---------------------------------------------------------|
| "25/08/2026" | -32602                     | Viola el <code>pattern</code> del esquema               |
| "2026-02-30" | -32602                     | Tiene la forma correcta pero no existe en el calendario |
| "2020-01-01" | <code>isError: true</code> | Argumento válido, lo rechaza una regla de negocio       |

## 8.7 Ejemplos de intercambio

Todos capturados de un servidor en ejecución.

**Error de dominio** — la cuenta no existe. El intercambio fue exitoso:

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "method": "tools/call",
  "params": {
    "name": "lookup_account",
    "arguments": {
      "account_id": "GT-10231"
    }
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "\n  \"error\": \"No se e"
      }
    ]
  }
}
```

**Error de protocolo** — falta un argumento obligatorio:

```
{
  "jsonrpc": "2.0",
  "id": 6,
  "method": "tools/call",
  "params": {
    "name": "check_service_status",
    "argume"
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 6,
  "error": {
    "code": -32602,
    "message": "account_id: is required",
    "data": {
      "account_id": "GT-10231"
    }
  }
}
```

## 8.8 Ejecución y conexión

Desde la raíz del repositorio:

```
python -m servers.netops.stdio_server
```

Desde el host propio del proyecto, que ya lo declara en `config/servers.json`:

```
python -m host.main
/call netops__lookup_account {"account_id": "GT-10231"}
```

El host aplica espacio de nombres a las herramientas como `<servidor>__<tool>`. El separador es doble guion bajo y no punto ni dos puntos, porque los nombres de herramienta de la API de Anthropic deben cumplir `^[a-zA-Z0-9_-]{1,128}$`.

**Figura 3.** El host propio imprime cada mensaje JSON-RPC enviado y recibido, que es el requisito de logging del enunciado. Se distingue el handshake completo — `initialize`, su resultado con el campo `instructions`, la notificación `notifications/initialized` sin respuesta, y `tools/list` — junto con las líneas de `stderr` del servidor, marcadas como tales y nunca confundidas con protocolo.

Desde Claude Code:

```
claude mcp add netops -- python -m servers.netops.stdio_server
claude mcp get netops
```

## 8.9 Persistencia

El almacenamiento separa lo inmutable de lo mutable, que es lo que mantiene los resultados reproducibles.

| Entidad           | Ubicación                            | Mutable                              |
|-------------------|--------------------------------------|--------------------------------------|
| Cuentas           | <code>data/seed/accounts.json</code> | No — versionado, solo lectura        |
| Incidencias       | <code>data/seed/outages.json</code>  | No — versionado, solo lectura        |
| Tickets y visitas | <code>data/state.json</code>         | Sí — escritura atómica, fuera de git |

La escritura del estado se hace mediante archivo temporal seguido de `os.replace`, que es atómico: una interrupción a medio camino no puede dejar el archivo truncado. Una prueba verifica que no queden archivos temporales huérfanos, y otra que la semilla nunca se escriba.

# 10. Conclusiones y comentarios sobre el proyecto

## 10.1 Estado actual

| Métrica                  | Valor                                                 |
|--------------------------|-------------------------------------------------------|
| Commits                  | 9, uno por fase más los de documentación y validación |
| Líneas de código fuente  | 2,416 (host 1,389 · servidores 873 · utilidades 154)  |
| Líneas de pruebas        | 1,466                                                 |
| Funciones de prueba      | 128, que ejecutan 160 casos por parametrización       |
| Herramientas expuestas   | 7                                                     |
| Dependencias de terceros | 4, ninguna relacionada con MCP                        |

Las seis fases planificadas (F0 a F5) están completas. El único criterio de aceptación no verificado es el de F4, por una razón externa al código que se detalla en 10.4.

## 10.2 Decisiones de diseño que resultaron acertadas

**Separar el transporte de la semántica.** El transporte mueve mensajes completos y no sabe nada de MCP; `MCPClient` es dueño de la correlación y del handshake. Cuando el proyecto avance a HTTP, la clase nueva solo tiene que implementar cuatro métodos, y nada del cliente cambia. La misma idea aplica del lado del servidor: `core.py` no conoce JSON-RPC, y `stdio_server.py` es apenas un adaptador. Añadir `http_server.py` no moverá lógica de negocio.

**Escribir `jsonrpc.py` como módulo puro, sin E/S.** Eso permitió probarlo por completo antes de que existiera un solo subproceso, y es la razón por la que F1 precedió a F2. Cuando aparecieron problemas de concurrencia en F2, se sabía con certeza que los sobres no eran la causa.

**Métricas determinísticas.** Derivarlas de un hash en lugar de `random` significa que una demostración puede repetirse y dar lo mismo, y que las pruebas no necesitan tolerancias.

**Un servidor de prueba que no comparte código con el cliente.** Las pruebas de F2 levantan un servidor ficticio que usa `json` pelado en lugar de nuestro propio codificador. Si ambos extremos compartieran el codificador, un error en él se cancelaría solo y la prueba pasaría igual.

## 10.3 Dificultades encontradas

---

**La concurrencia del transporte fue lo más costoso**, tal como anticipaba el enunciado. El problema no fue escribir el hilo lector sino los caminos de falla: si el servidor muere, cada petición pendiente debe fallar con una excepción en lugar de quedar colgada para siempre. Eso se resolvió fallando explícitamente todos los `Future` pendientes al detectar EOF, y hay pruebas que matan al servidor a propósito y exigen que el llamador reciba un error.

**La codificación en Windows** exigió forzar UTF-8 tanto en el proceso hijo como en los flujos del servidor. El valor por defecto `cp1252` corrompe cualquier carga útil con acentos, y los datos semilla contienen varios.

**Dos fallas de conformidad aparecieron solo al manejar el servidor desde afuera.** Conducirlo desde PowerShell, en lugar de desde nuestro propio host, reveló lo que las 151 pruebas existentes hasta ese momento no detectaban, porque host y servidor compartían supuestos:

1. `notifications/initialized` marcaba la sesión como inicializada aunque `initialize` nunca hubiera corrido, de modo que un cliente podía saltarse el handshake y aún así alcanzar `tools/list`. La notificación *confirma* un handshake; no puede iniciarlo.
2. Una marca de orden de bytes al inicio de la línea hacía fallar el parseo con `-32700`. JSON es UTF-8 y la marca sobra, pero varias herramientas la emiten y el RFC 8259 permite ignorarla.

Ambas tienen prueba de regresión. La lección es que las pruebas escritas contra la propia implementación tienen un punto ciego estructural: solo un cliente ajeno lo revela.

**Conectar Claude Desktop en Windows tuvo dos obstáculos no documentados.** El primero es que la instalación desde Microsoft Store empaqueta la aplicación como MSIX, lo cual virtualiza `%APPDATA%`: el archivo de configuración que la aplicación lee realmente no está en `%APPDATA%\Claude\` sino en `%LOCALAPPDATA%\Packages\Claude_pzs8sxrjxfjjc\LocalCache\Roaming\Claude\`, de modo que escribir

en la ruta documentada no surte ningún efecto. El segundo es que cerrar la ventana no recarga la configuración, porque la aplicación queda residente; un segundo lanzamiento detecta la instancia viva y termina de inmediato registrando `Not main instance, returning early from app ready`. Hay que salir desde el ícono de la bandeja o terminar todos los procesos. Ambos detalles quedaron documentados en `SPEC.md` sección 7.5 para no tener que redescubrirlos.

## 10.4 Lo que falta

---

**El bucle agéntico del host propio no se ha ejecutado contra la API real.** La clave de API autentica correctamente, pero la cuenta no tiene saldo, y la única llamada real intentada devolvió `400: credit balance is too low`. El bucle está cubierto por pruebas que lo conducen con un modelo guionado contra los servidores reales, lo cual valida el ruteo, la traducción de esquemas y el manejo de errores, pero no valida que ese bucle en particular haya conducido a un modelo real.

Conviene precisar el alcance de lo que falta, porque es más estrecho de lo que parece. Que **un modelo real invoque las herramientas** sí está demostrado: se verificó desde Claude Desktop, como se documenta en 10.5. Lo que no está demostrado es que lo haga a través de `host/agent.py`, es decir, del bucle escrito en este proyecto. La lógica del servidor, los esquemas y el protocolo quedan validados por esa prueba; el código pendiente de ejercitar contra un modelo real son las aproximadamente ciento cincuenta líneas del agente propio.

Queda también pendiente todo lo correspondiente al proyecto final: el transporte HTTP remoto, el despliegue con Docker y Cloud Run, la captura con Wireshark y la integración de servidores oficiales de terceros.

## 10.5 Validación con un host de terceros

---

El enunciado sugiere conectar el servidor a **Claude Desktop** como prueba de conformidad. Se realizó, contra la versión 1.32885.1.0, y el resultado quedó registrado en los logs de ambos extremos.

Del lado del host:

```
[LocalMcpServerManager] netops negotiated protocol version: 2025-11-25
[LocalMcpServerManager] Connected to netops (7 tools)
[localMcpBridge] announcing netops: 7 tool(s)
```

Del lado del servidor, en el `stderr` que el propio host capturó:

```
[netops] listening on stdio, protocol 2025-11-25
[netops] initialize from claude-ai 0.1.0
[netops] handshake complete
```

La línea `initialize from claude-ai 0.1.0` es significativa: no es un mensaje genérico sino el servidor leyendo el campo `clientInfo` que Claude Desktop envió en el handshake. Es la implementación propia procesando datos reales de un cliente ajeno.

**Figura 1.** Al preguntársele qué herramientas tiene disponibles, el modelo enumera las siete con sus descripciones, que son exactamente las que el servidor publica en `tools/list`. Nótese la entrada de `open_ticket`: el modelo cita la guía del propio servidor —revisar `list_outages` primero— sin que nadie se la haya mencionado en la conversación. Esa guía llegó por el campo `instructions` del handshake.

**Invocación de herramientas por un modelo real.** Más allá del handshake, se verificó el uso efectivo. Ante la consulta *"revisá el estado de la cuenta GT-10233 y decime si conviene abrir ticket"*, el modelo encadenó tres herramientas por su cuenta —`lookup_account`, `check_service_status` y `list_outages`— y concluyó:

*No abrir ticket individual. El corte ya está cubierto por el outage masivo, así que un ticket nuevo para esta cuenta solo generaría ruido y no acelera nada.*

**Figura 2.** El modelo decide por su cuenta qué herramientas invocar y en qué orden. Primero `lookup_account`, luego dos más para el estado del enlace y las incidencias de la región, y sobre esos resultados construye la recomendación de no abrir ticket. También advierte que el ETA de la incidencia ya venció mientras su estado sigue activo, una inconsistencia de los datos que ninguna instrucción le pidió buscar.

Esa conclusión no vino de la pregunta del usuario ni de conocimiento previo del modelo: vino del campo `instructions` que el servidor devuelve en `initialize`, donde se indica que ante un reclamo de conectividad debe consultarse `list_outages` primero, porque si una incidencia masiva ya cubre la región el ETA es la respuesta y un ticket nuevo solo agrega ruido. El servidor no se limitó a exponer datos: orientó el comportamiento del modelo, y el modelo siguió esa orientación.

El modelo además detectó una inconsistencia en los datos: el ETA de la incidencia ya había vencido mientras su estado seguía marcado como activo. Es un comportamiento realista en el dominio —los cortes se pasan del tiempo estimado con frecuencia— y el modelo lo señaló en lugar de ignorarlo.

**Validaciones adicionales.** El servidor también conecta desde **Claude Code** (`claude mcp get netops reporta Status: ✓ Connected`), y se escribió `tools/conformance_check.py`, un cliente independiente que usa solo la biblioteca estándar y no importa nada de `host/`, que verifica 19 aserciones sobre el ciclo de vida, las herramientas, los errores y el apagado. Tres hosts distintos, dos de ellos de producción y ninguno con conocimiento del código de este repositorio.

## 10.6 Comentario sobre MCP

---

Lo más valioso de implementar el protocolo a mano fue entender que MCP resuelve un problema de acoplamiento más que uno técnico. El protocolo en sí es JSON-RPC 2.0 con un handshake y tres métodos; nada de eso es difícil. Lo que aporta es que el servidor publica sus propias capacidades —nombres, esquemas y hasta instrucciones dirigidas al modelo— de modo que el host no necesita saber nada sobre el dominio del servidor, y el servidor no necesita saber qué modelo lo va a usar. Escribir el cliente y el servidor en el mismo repositorio y luego ver a Claude Code conectarse al servidor sin modificar una línea deja esa idea clara de una forma que leer la especificación no logra.

La distinción entre error de protocolo y error de herramienta fue el punto donde más se aprendió. Es tentador tratar "la cuenta no existe" como un error, y el protocolo insiste en que no lo es: es un resultado exitoso con contenido que el modelo debe leer. Esa separación es lo que permite que un modelo se recupere solo de un argumento equivocado, y es la razón por la que el bucle agéntico devuelve cada falla de herramienta al modelo en lugar de propagarla como excepción.